

Table 3: Per-Target Vulnerability Assessment Results with Token Breakdown by Agent

Target	GitHub ★	Main Agent Tokens			Sandbox Agent Tokens			Vulnerabilities			Cost (\$)
		Regular	Cached	Output	Regular	Cached	Output	H	M	L	
OSN-06	21K	22K	270K	12K	322K	6.9M	70K	4	2	0	4.85
OSN-03	9K	9K	17K	11K	28K	372K	23K	5	1	0	1.57
OSN-04	18K	47K	834K	15K	176K	1.1M	117K	1	1	1	6.05
OSN-05	36K	40K	615K	20K	253K	1.7M	116K	2	0	0	6.55
OSN-01	26K	221K	3.8M	18K	182K	200K	180K	1	0	0	8.02
OSN-02	8K	8K	18K	8K	79K	657K	30K	1	0	0	1.97
appsmith	38K	12K	35K	9K	40K	339K	34K	0	0	0	2.11
directus	32K	11K	58K	11K	40K	536K	34K	0	0	0	1.97
gitea	50K	9K	18K	9K	131K	1.4M	27K	0	0	0	1.93
grafana	70K	7K	25K	10K	254K	432K	19K	0	0	0	1.73

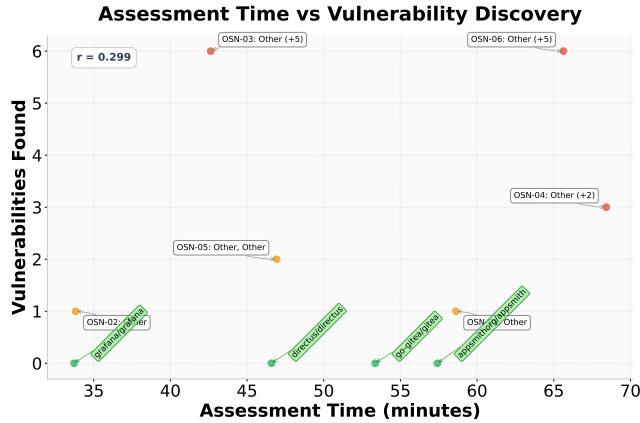


Figure 10: Assessment time versus vulnerability discovery patterns. Labels indicate the types of vulnerabilities found.

4.1 Vulnerability Discovery Results

Responsible Disclosure Note: In accordance with responsible disclosure practices, we have anonymized the identities of applications where vulnerabilities were discovered, using obfuscated names (OSN-XX) for these targets. Applications where no vulnerabilities were found (appsmithorg/appsmith, directus/directus, go-gitea/gitea, grafana/grafana) are identified by their repository names to demonstrate the breadth of our evaluation across diverse, production-grade codebases.

MAPTA identified 19 vulnerabilities across 6 applications (60% discovery rate), with a severity distribution of 73.7% High/Critical, 21.1% Medium, and 5.3% Low/Informational. Assessment costs averaged \$3.67 per application over 50.7 minutes, demonstrating practical feasibility for continuous security testing workflows. Figure 9 illustrates the relationship between vulnerability discovery and assessment costs across target applications, showing that cost does not directly correlate with findings—some of the most expensive assessments

yielded no vulnerabilities while others discovered critical issues at lower computational cost.

Example Critical Vulnerabilities Discovered:

- **Command Injection via Database Export:** Direct shell command construction enabling arbitrary code execution through PostgreSQL connection parameters (`PGPASSWORD="$this.config.password" pg_dump -schema-only "$input"`)
- **Client-Side Secret Exposure:** Server-side API keys delivered via JavaScript configuration endpoints (`window.env = {OPENAI_API_KEY: "$OPENAI_API_KEY"}`)
- **postMessage RCE:** Arbitrary code execution through overly permissive cross-frame origin validation (`case 'builder.evaluate': new Function(text)`)
- **Unauthenticated Email Relay with SSRF:** Public API endpoints accepting arbitrary SMTP credentials and remote attachment URLs (`fileUrls: "http://169.254.169.254/latest/meta-data/"`)
- **Arbitrary File Write via Client-Controlled Tools:** Remote clients enabling dangerous file operations through tool merging (`input.tools.override` enabling `PatchTool`)

Example High Severity Patterns:

- **Unauthenticated API Integration Abuse:** Third-party service access using attacker-supplied credential IDs (Google Sheets, Stripe PaymentIntent creation)
- **Insecure Cryptographic Implementation:** Non-cryptographic RNG for API key generation (`Math.random()` for 64-character secret keys)
- **Path Traversal via File Access APIs:** Unvalidated file path parameters enabling arbitrary file reads (`File.read(path)` without containment checks)

- **Unauthenticated Administrative Endpoints:** Critical system operations exposed without authorization (`/share_delete_admin` clearing Durable Objects)

Example Medium Severity Patterns:

- **XSS via Environment Injection:** Unescaped server-side template rendering in configuration endpoints (`"${OPENAI_API_ENDPOINT}"` string interpolation)
- **CSRF Across REST APIs:** State-changing operations without Origin validation or CSRF tokens (API token creation, user invitations)
- **SSRF via Integration APIs:** Server-side request forgery through legitimate webhook and file import functionality
- **Open Redirect via Payment Flows:** Unchecked URL parameters in checkout processes (`success_url`, `cancel_url`)

5 Related Work

5.1 Classical Automated Web Security Testing

Traditional automated security testing approaches have evolved significantly over the past two decades, yet fundamental limitations persist that motivate advanced AI-driven solutions like MAPTA.

Dynamic scanners like OWASP ZAP [20] and Burp Suite [22] crawl web applications and fuzz HTTP parameters to identify common vulnerabilities. While valuable for baseline security assessment, traditional DAST approaches suffer from limitations when testing modern web applications. Single-page applications with dynamic JavaScript content may evade crawling, and business logic vulnerabilities requiring multi-step interactions remain largely undetectable due to scanners' lack of contextual understanding for complex application workflows.

Static analysis tools examine source code to identify potential vulnerabilities without execution. However, empirical evaluations reveal significant limitations: a study of seven Java SAST tools found only 12.7% of real-world vulnerabilities were detected, with the union of all tools missing 71% [16]. Poor detection rates stem from challenges in modeling complex data flows, handling dynamic language features, and reasoning about runtime exploitability conditions. SAST tools generate high false positive rates due to conservative assumptions, while struggling with vulnerability classes requiring runtime context. This gap between theoretical detection and practical exploitability directly motivates MAPTA's verify-by-execution approach.

Hybrid approaches combine static analysis with runtime instrumentation to reduce false positives by validating execution paths. However, deployment challenges limit adoption

due to instrumentation requirements, performance overhead, and complexity across microservices and containers.

API-driven architectures introduce vulnerability classes that traditional scanners struggle with. The OWASP API Security Top 10 (2023) [17] highlights business logic vulnerabilities like BOLA, BFLA, and IDOR requiring understanding of application-specific access controls. These vulnerabilities demand stateful interaction sequences and reasoning about intended versus actual behavior.

5.2 Stateful REST/API Fuzzing

Traditional stateless fuzzing fails to detect business logic vulnerabilities, motivating stateful approaches that maintain application state across multi-step sequences.

Microsoft Research's RESTler [3] introduced request dependency graphs from OpenAPI specifications, analyzing relationships between API calls to construct meaningful multi-step interaction sequences. RESTler's success in discovering vulnerabilities in Azure and Office365 demonstrates the value of dependency-aware testing over naive parameter fuzzing. Extensions like Pythia [2] add coverage feedback and learning-based mutations for more targeted exploration.

Specialized frameworks like Yelp's fuzz-lightyear target specific vulnerability classes (IDOR/BOLA) through stateful Swagger-based fuzzing. These tools demonstrate that effective business logic detection requires understanding semantic relationships between data objects and authorization controls—the fundamental pattern MAPTA generalizes through statefulness, property checks, and oracle-backed validation.

5.3 LLMs for Secure Code

Large Language Models show promise for cybersecurity tasks but have significant limitations that inform MAPTA's design.

GitHub Copilot generates code containing vulnerabilities in 40% of CWE-targeted scenarios [21], stemming from reproducing insecure patterns in training data. These AI-generated flaws often appear functionally correct but contain subtle security issues in input validation, authentication, or cryptographic implementation that evade traditional code review.

Comprehensive surveys [6] show that while LLMs excel at security reasoning and hypothesis generation, they require external oracles and environment feedback to validate outputs and avoid hallucinations—a pattern MAPTA addresses through tool integration and concrete execution.

Google's Big Sleep project discovered a zero-day in SQLite (November 2024) and helped foil exploitation [11, 12]. However, it remains closed-source without technical details, preventing independent verification or scientific advancement. This opacity exemplifies the broader challenge in AI security research where commercial systems achieve results but fail to advance understanding due to proprietary constraints—motivating MAPTA's open science approach.

5.4 LLM-Driven Autonomous Testing and Tool Orchestration

Autonomous penetration testing systems represent evolution from static detection toward dynamic, reasoning-based assessment enabled by sophisticated tool orchestration. Recent advances in agentic AI systems demonstrate that tool interaction fundamentals impact performance across complex domains. ReAct [28] and Toolformer [24] established that LLMs achieve superior performance through structured tool interaction and environmental feedback loops, while SWE-agent [27] demonstrates that interface design and tool abstractions determine success rates on complex technical tasks.

PentestGPT [8] pioneered multi-stage LLM workflows for enumeration, exploitation, and privilege escalation with self-interaction capabilities. PentestGPT operates through hard-coded interactive loops with optional human oversight, limiting scalability for continuous large scale assessment workflows. Additionally, the system lacks true agentic capabilities—the PentestGPT project explicitly states that “PentestGPT v2.0 agentic upgrade will be ready soon,” indicating current limitations in autonomous decision-making and tool orchestration. While contributing structured prompting techniques and evaluation metrics, the system revealed limitations in long-horizon state management and vulnerability validation. The system reports aggregate costs (\$131.5 for 10 HTB machines; \$5.1 average per picoMini attempt) and discusses token conservation strategies with GPT-4-32k context limits.

Subsequent research addresses these limitations through complementary approaches: PenHeal [13] couples discovery with remediation using knapsack optimization but does not report token usage—the “cost” metric represents remediation scoring rather than LLM operational expenses. RefPenTester [7] adds self-reflection and knowledge-guided planning, while browser-capable agents [15] enable direct web interaction for CSRF/SSRF testing.

MAPTA advances autonomous security assessment through resource measurement and operational efficiency analysis that addresses fundamental gaps in prior work. Our evaluation provides complete token-level accounting across 104 XBOW challenges: 3.2M regular input, 1.10M output, 50.5M cached, and 0.595M reasoning tokens, totaling \$21.38 overall cost with median \$0.117 per challenge. This granular breakdown reveals output tokens as the primary cost driver, enabling resource optimization strategies unavailable prior.

Beyond cost accounting, MAPTA quantifies negative correlations between resource utilization and success—tool calls ($r=-0.661$), dollar cost ($r=-0.606$), tokens ($r=-0.587$), and time ($r=-0.557$)—providing actionable early-stopping heuristics and budget allocation guidance for production deployments. Our multi-agent architecture employs a coordinator/sandbox design with dynamic tool use, combined with end-to-end proof-of-concept validation that eliminates the false positives inherent in theoretical detection approaches. While prior sys-

tems discuss token pressure mitigation strategies, MAPTA measures and quantifies the complete operational profile, establishing the first rigorous cost-performance framework for autonomous penetration testing systems.

5.5 Benchmarks and Testbeds

Traditional vulnerable applications (Juice Shop [18], Web-Goat [19], DVWA [9]) focus on vulnerability types with implementations unsuitable for evaluating advanced systems. The XBOW benchmark dataset [25] represents significant advancement by providing modern web application challenges with REST APIs, complex business logic, and realistic authentication mechanisms. XBOW’s key innovation emphasizes exploit execution validation over theoretical detection—each challenge requires actual exploitation success, eliminating false positives and aligning with real-world penetration testing objectives. Our approach builds on the fundamental insight from related work that effective automated security assessment requires tool orchestration, stateful reasoning, and practical verification [3, 28]. MAPTA’s multi-agent architecture with sandboxed exploit validation directly addresses the limitations identified in single-agent systems like PentestGPT [8] and traditional scanners’ false-positive challenges [16].

6 Conclusion

MAPTA demonstrates that multi-agent architectures can achieve competitive autonomous web application security assessment at practical scale. Our evaluation across 104 XBOW challenges achieves 76.9% success with perfect performance on SSRF and misconfiguration vulnerabilities, while revealing systematic weaknesses in blind SQL injection (0%) and cross-site scripting (57%). The comprehensive cost accounting totaling \$21.38 establishes the first rigorous resource model for autonomous penetration testing, with median costs of \$0.073 for successful attempts versus \$0.357 for failures.

While our CTF evaluation ($N=104$) revealed strong correlations between resource usage and success (enabling early-stopping thresholds at approximately 40 tool calls, \$0.30, or 300 seconds), these patterns cannot be validated in our whitebox assessment due to the smaller sample size ($N=10$). Yet, MAPTA’s real-world validation is impactful with 19 discovered vulnerabilities across ten popular open-source applications, of which *14 classified as high or critical severity* (including RCE, command injections, secret exposure and arbitrary file write), at an average cost of \$3.67 per assessment. All findings are responsibly disclosed to the respective parties and bug bounty programs, where applicable. In total we are awaiting responses from 10 findings that are under CVE review. We expect that larger real-world scans will uncover substantially more vulnerabilities, and recommend deploying MAPTA on a continuous basis for immediate defensive action of web applications.

Ethical Considerations

The development and evaluation of MAPTA raises important ethical considerations regarding responsible disclosure of AI-powered security testing capabilities. We address these concerns through several key principles and safeguards implemented throughout our research.

Defensive Publication and Community Awareness. The primary ethical imperative for publishing this research stems from the reality that adversarial actors likely possess similar capabilities or are actively developing them. The democratization of AI development tools and the public availability of security testing methodologies means that malicious applications of these techniques are inevitable. By publishing our findings, we enable the cybersecurity community to understand and prepare for these emerging threats. Defensive security benefits from transparency about offensive capabilities, allowing organizations to implement appropriate countermeasures and security professionals to develop detection and mitigation strategies.

Controlled Evaluation Environments. Our evaluation methodology deliberately avoids testing against live production systems to prevent unintended harm or service disruption. We conducted two distinct types of assessments: (1) black-box evaluation using purpose-built CTF challenges from the XBOW benchmark, which are explicitly designed for security testing and vulnerability discovery, and (2) whitebox assessments of open-source applications conducted entirely within isolated local environments. The whitebox evaluations involved cloning public repositories and conducting all testing within our own sandboxed virtual machines, ensuring no impact on production deployments or third-party infrastructure.

Sandboxed Testing Infrastructure. We implemented isolation measures to prevent any testing activities from affecting external systems. All MAPTA evaluations execute within dedicated virtual machines with restricted network access, preventing unintended outbound connections or data exfiltration. The sandbox environment includes monitoring and logging mechanisms to ensure all testing activities remain contained within the designated test boundaries. This approach eliminates risks of collateral damage while maintaining the authenticity of real-world vulnerability assessment scenarios.

Responsible Vulnerability Disclosure. For vulnerabilities discovered during our whitebox assessments, we follow responsible disclosure practices by notifying maintainers of affected projects through appropriate channels. The 10 vulnerabilities submitted for CVE assignment were reported to the respective project maintainers with sufficient detail for remediation while avoiding public disclosure of exploitation techniques until patches are available. We provide actionable remediation guidance and collaborate with maintainers to ensure timely resolution of identified security issues.

Dual-Use Technology Considerations. We acknowledge that MAPTA represents dual-use technology with both defen-

sive and potentially offensive applications. To mitigate misuse risks, our implementation focuses on defensive security applications and includes built-in ethical constraints that prevent destructive operations, data exfiltration, or persistent system modifications. The system is designed to generate proof-of-concept demonstrations rather than weaponized exploits, providing sufficient evidence for vulnerability validation without enabling direct malicious use.

Access Control and Distribution. While we commit to making MAPTA source code publicly available upon publication to enable scientific reproducibility and defensive research, we implement responsible access controls. The release includes documentation emphasizing ethical use guidelines, configuration options for defensive-only operation modes, and integration with existing responsible security testing frameworks. We encourage adoption by legitimate security professionals, researchers, and organizations while discouraging malicious applications through community governance and ethical use agreements.

The fundamental ethical principle guiding this research is that the cybersecurity community benefits more from understanding these capabilities than from attempting to suppress them. As AI-powered development accelerates application creation, correspondingly advanced security assessment tools become essential for maintaining adequate security postures. MAPTA represents a defensive response to this challenge, providing organizations with capabilities to match the evolving threat landscape while adhering to responsible research and deployment practices.

Open Science & Availability

In accordance with the Open Science Policy, we provide complete access to all research artifacts necessary to evaluate and reproduce the contributions presented in this paper. All artifacts are available at <https://github.com/arthurgervais/mapta>. The updated XBOW 104 Challenge Evaluation Framework is available at <https://github.com/arthurgervais/validation-benchmarks>.

References

- [1] Waleed Alasmary, Feras Khan, Ghada Almashaqbeh, et al. A survey of business logic vulnerabilities in web applications. *Information*, 16(7):585, 2025.
- [2] Vaggelis Atlidakis, Roxana Geambasu, Patrice Godefroid, Marina Polishchuk, and Baishakhi Ray. Pythia: Grammar-based fuzzing of rest apis with coverage-guided feedback and learning-based mutations. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020.

- [3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. Restler: Stateful rest api fuzzing. In *International Conference on Software Engineering (ICSE)*, 2019.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in neural information processing systems*, volume 33, pages 1877–1901, 2020.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.
- [6] Xiaozhu Chen, Yuhang Zhou, Zihan Wang, et al. Large language models for cyber security: A systematic literature review. *arXiv preprint arXiv:2405.04760*, 2024.
- [7] Hanzheng Dai, Yuanliang Li, Zhibo Zhang, and Jun Yan. Refpentester: A knowledge-informed self-reflective penetration testing framework based on llms, 2025.
- [8] Gelei Deng, Ziniu Hu, Yueqi Chen, Haoyu Wang, Bangjie Yin, Yinzhi Cao, Gang Wang, Yan Chen, Xinyu Xing, and Zhiqiang Lin. Pentestgpt: Evaluating and harnessing large language models for automated penetration testing. In *USENIX Security*, 2024.
- [9] Ryan Dewhurst. Damn vulnerable web application (dvwa), 2025.
- [10] Brendan Dolan-Gavitt. Ai agents for offsec with zero false positives, 2025.
- [11] Google Cloud CISO Office. Our big sleep agent makes a big leap. Google Cloud Blog, 2025.
- [12] Google Project Zero. From naptime to big sleep: Using large language models to find real-world vulnerabilities. Project Zero Blog, 2024.
- [13] Junjie Huang and Quanyan Zhu. Penheal: A two-stage llm framework for automated pentesting and optimal remediation. In *Proceedings of the ACM Conference Companion on Computer and Communications Security (ACM CCS Companion), AutonomousCyber '24: Proceedings of the Workshop on Autonomous Cybersecurity*, 2024.
- [14] Imperva. Business logic attacks: Why traditional tools fall short. <https://www.imperva.com/blog/business-logic-attacks-traditional-tools-shortcomings/>, 2023. Accessed: 2025-08-21.
- [15] N. Kalopisis. Browser-empowered llm agents for web penetration testing. Master’s thesis, University of Twente, 2025.
- [16] Kaixuan Li, Sen Chen, Lingling Fan, Ruitao Feng, Han Liu, Chengwei Liu, Yang Liu, and Yixiang Chen. Comparison and evaluation on static application security testing (sast) tools for java. In *ESEC/FSE*, 2023.
- [17] OWASP Foundation. Owasp api security top 10: 2023, 2023.
- [18] OWASP Foundation. Owasp juice shop, 2025.
- [19] OWASP Foundation. Owasp webgoat, 2025.
- [20] OWASP ZAP Project. Zed attack proxy (zap) documentation, 2025.
- [21] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768. IEEE, 2022.
- [22] PortSwigger Ltd. Burp suite documentation, 2025.
- [23] Positive Technologies. Web application vulnerabilities in 2020–2021. <https://global.ptsecurity.com/en/research/analytics/web-vulnerabilities-2020-2021/>, 2021. Accessed: 2025-08-21.
- [24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- [25] XBOW Engineering. Xbow validation benchmarks. <https://github.com/xbow-engineering/validation-benchmarks>, 2024. Accessed: 2024-12-01.
- [26] XBOW Engineering. Gpt-5 performance analysis for autonomous penetration testing. XBOW Blog, 2025. Accessed: 2025-01-26.
- [27] John Yang, Carlos E. Jimenez, Ofir Press, and Karthik Narasimhan. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022.